

*Heaven's Light is Our Guide*

## Rajshahi University of Engineering & Technology



Department of Computer Science & Engineering  
24 Series

Course No.: 1201  
Course Title: Data Structure

**Assignment No. : 01**

**Assignment Title : Understanding Fundamental Data Structures  
and Their Computational Concepts**

**Submitted by:**

Fardin Bin Aslam Numan

Roll: 2403179

Section: C

Session: 2024-25

**Submitted to:**

Dr. Md. Nazrul Islam Mondal

Professor

Dept. of CSE, RUET

Emrana Kabir Hashi

Assistant Professor

Dept. of CSE, RUET

**Date of Submission : 28 November 2025**

# Banking System

## Objectives:

- To analyze how fundamental data structures support the internal operations of an ATM-based banking system
- To explain how correct structure selection improves the system's speed, reliability, and resource management

## Background:

A banking system maintains sensitive and a huge amount of data, which includes account records, transaction histories (withdrawals and deposits), verification processes and customer requests. For faster verification, secure access, and smooth transaction processing, efficient data structures are necessary, as in peak times, thousands of users access the ATMs simultaneously and performance directly impacts transaction speed and user satisfaction

## Algorithm:

1. Start
2. Input user's account number
3. Search the account in the Hash Table
4. If account not found >> Display "Invalid Account" >> Stop
5. Input user PIN
6. If PIN incorrect >> Display "Wrong PIN" >> Stop
7. Insert the request into the ATM Queue
8. Wait until the request reaches the front of the queue
9. Input user's selected option
10. If Withdraw selected >> Check if balance is sufficient
11. If sufficient >> Deduct amount | Else >> Display "Insufficient Balance"
12. If Deposit selected >> Add deposit amount to balance
13. If Check Balance selected >> Display current balance
14. Record the transaction in the Doubly Linked List (insert node at end)
15. Update modified account information in the Hash Table
16. Display "Transaction Successful"
17. Stop

Table: Functionalities and Suitable Data Structures with Justifications

| Functionality                                              | Data Structure     | Justifications                                                                                                                                               |
|------------------------------------------------------------|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Storing account information (Account number, balance, PIN) | Hash Table         | Hash Table enables constant-time search without scanning the entire database for account verification using account number as a key                          |
| Managing ATM transaction queue                             | Queue              | It ensures "First Come First Served" processing for ATM users to avoids request collisions to maintain fairness and supports real time incoming transactions |
| Recording user transaction history                         | Doubly Linked List | Allows efficient insertion of new transaction records at the end while supporting backward and forward traversal                                             |

### Discussion on Improper Selection:

- Using an **array** instead of a **hash table** would require scanning all accounts for every verification which would eventually slow response time
- Using a **stack** instead of a **queue** for ATM requests would reverse the service order and disrupt fairness
- Using an **array** instead of a **linked list** for transaction logs increases insertion cost and wastes memory

### Application & Reflection:

In a banking system, data structures directly influence its speed and stability. Activities such as account verification, withdrawal processing and transaction logging have to be performed in real time and consistently. Implementing a **hash table** for account records makes verification almost instantaneous because the mapping between the account number and record eliminates superfluous searching. The **queue** structure guarantees that users at the ATM are processed in the exact order in which they come, maintaining fairness and preventing overlap of requests. The histories of transactions continue to grow and a **doubly linked list** supports efficient insertion and smooth navigation across the records without reallocation

### Conclusion:

A banking system runs efficiently when each component uses the appropriate data structure. Correct choices support speed, fairness and system stability. This assignment demonstrated that good system performance comes from choosing structures aligned with the type and pattern of data being processed